# ENSF-381: Assignment 4

Winter 2026

Department of Electrical and Software Engineering

Schulich School of Engineering

Due: April 10, 2026, at 11:59 PM

## Objective

In this assignment, you will extend your Sweet Scoop React application from Assignment 3 into a simple full-stack web application. You will move your application data to the backend, implement user registration and login, and connect the frontend order flow to backend APIs.

## Administrative Details

- The submission should be a single compressed file (.zip) that includes the complete project needed to run both the frontend and backend.
- Name the submitted file as: Assignment4_ENSF381_Group#. For example: Assignment4_ENSF381_L02_Group01.zip
- Submit only one copy per group.
- Include group members' names and UCIDs as a docstring at the top of app.py.

## Backend Setup

Create a new directory named backend at the project root. Inside backend, create app.py for your Flask application.

Create a virtual environment for the backend and install the following dependencies:

- Flask
- flask-cors
- bcrypt

The React frontend should communicate with the Flask backend through HTTP requests.

Move the Assignment 3 data files into backend JSON files:

- Copy the flavor data from src/data/flavors.js into backend/flavors.json.
- Copy the review data from src/data/reviews.js into backend/reviews.json.

You do not need to serve the flavor image files from the backend for this assignment. Keep the actual image files in the frontend public/images/ folder, and store only their path strings in backend/flavors.json (for example, images/strawberry.jpg).

# Backend User Data

Maintain a backend user structure in memory using a Python list or dictionary. Each user must include at least an id, username, email, password_hash hashed with bcrypt, a cart, and an orders history.

*You may pre-populate the backend with a few sample users for testing. If you do, their password_hash values must already be valid bcrypt hashes so that the login flow can verify them correctly.*

Your cart data should store enough information to render the frontend order list, including flavorId, name, price, and quantity.

Each stored order should include enough information to support an Order History page, such as orderId, items, total, and a timestamp or date.

Example user object:

```
{
 "id": 1,
 "username": "sweet_alice",
 "email": "alice@example.com",
 "password_hash": "$2b$12$examplehashedvalue",
 "cart": [
   {
    "flavorId": 2,
    "name": "Chocolate Bliss",
    "price": 5.49,
    "quantity": 2
   }
 ],
 "orders": [
   {
    "orderId": 1,
    "items": [
      {
       "flavorId": 1,
       "name": "Vanilla Dream",
       "price": 4.99,
       "quantity": 1
      }
    ],
    "total": 4.99,
    "timestamp": "2026-03-30 18:30:00"
   }
 ]
}
```

# API Requirements

## 1. Registration API

**Endpoint:** POST /signup

This endpoint creates a new user account. It should:

- Validate all signup data.
- Reject invalid usernames, emails, or passwords.
- Reject duplicate usernames and emails.
- Hash the password using bcrypt. Passwords must never be stored in plain text. See the MDN Passwords and the pyca/bcrypt repository.
- Store the bcrypt hash in the backend user record and use it again when verifying the password during login.
- Add the new user to the backend user structure.
- Return a success or error message.

*Validation requirements:*

- Username must be between 3 and 20 characters.
- Username must start with a letter.
- Username may contain letters, numbers, underscores, and hyphens only.
- Email must be in a valid email format.
- Password must be at least 8 characters long.
- Password must contain at least one uppercase letter, one lowercase letter, one number, and one special character.

Example request:

```
{
  "username": "sweet_alice",
  "email": "alice@example.com",
  "password": "IceCream!23"
}
```

Example response:

```
{
  "success": true,
  "message": "Registration successful."
}
```

Example error response:

```
{
  "success": false,
```

```
  "message": "Username is already taken."
}
```

## 2. Login API

**Endpoint:** POST /login

This endpoint authenticates an existing user. It should:

- Check whether the username exists.
- Compare the submitted password with the stored bcrypt hash.
- On success, return login data for the frontend session.
- On failure, return an error message that can be displayed on the frontend.

Example request:

```
{
  "username": "sweet_alice",
  "password": "IceCream!23"
}
```

Example success response:

```
{
  "success": true,
  "message": "Login successful.",
  "userId": 3,
  "username": "sweet_alice"
}
```

Example error response:

```
{
  "success": false,
  "message": "Invalid username or password."
}
```

## 3. Reviews API

**Endpoint:** GET /reviews

This endpoint returns two random reviews. It should:

- Read review data from reviews.json.
- Return 2 random reviews to the frontend.

Example request:

```
GET /reviews
```

Example response:

```
{
 "success": true,
 "message": "Reviews loaded.",
 "reviews": [
   {
    "customerName": "Ava",
    "rating": 5,
    "review": "Sweet Scoop is my favorite weekend treat."
   }
 ]
}
```

## 4. Flavors API

**Endpoint:** GET /flavors

This endpoint returns all available Sweet Scoop flavors. It should:

- Read flavor data from flavors.json.
- Return all available flavors to the frontend.

Example request:

```
GET /flavors
```

Example response:

```
{
 "success": true,
 "message": "Flavors loaded.",
 "flavors": [
   {
    "id": 1,
    "name": "Vanilla Dream",
    "price": "$4.99",
    "description": "Classic creamy vanilla ice cream made with fresh ingredients.",
    "duration": "8 weeks",
    "image": "images/vanilla.jpg"
   }
 ]
}
```

## 5. Get Cart API

**Endpoint:** GET /cart

This endpoint returns the current cart for the logged-in user. It should:

- Find the user by userId.
- Return that user's current cart.

Example request:

```
GET /cart?userId=3
```

Example response:

```
{
  "success": true,
  "message": "Cart loaded.",
  "cart": [
    {
      "flavorId": 2,
      "name": "Chocolate Bliss",
      "price": 5.49,
      "quantity": 1
    }
  ]
}
```

## 6. Add to Cart API

**Endpoint:** POST /cart

This endpoint adds one unit of a flavor to the logged-in user's cart. It should:

- Validate that the user exists.
- Validate that the flavor exists.
- If the flavor is not already in the cart, add it with quantity 1.
- If the flavor is already in the cart, return an error and require the frontend to use PUT /cart to update the quantity instead.
- Return the updated cart and a message.

Example request:

```
{
  "userId": 3,
  "flavorId": 2
}
```

Example response:

```
{
  "success": true,
  "message": "Flavor added to cart.",
  "cart": [
    {
```

```
    "flavorId": 2,
    "name": "Chocolate Bliss",
    "price": 5.49,
    "quantity": 1
  }
 ]
}
```

## 7. Update Cart Quantity API

**Endpoint:** PUT /cart

This endpoint sets the quantity of a cart item. It should:

- Validate that the user exists.
- Validate that the flavor exists in the user's cart.
- Validate that quantity >= 1.
- Update the cart item to the exact quantity provided.
- Return the updated cart and a message.

Example request:

```
{
 "userId": 3,
 "flavorId": 2,
 "quantity": 2
}
```

Example response:

```
{
 "success": true,
 "message": "Cart updated successfully.",
 "cart": [
  {
    "flavorId": 2,
    "name": "Chocolate Bliss",
    "price": 5.49,
    "quantity": 2
  }
 ]
}
```

## 8. Delete Cart Item API

**Endpoint:** DELETE /cart

This endpoint removes a flavor from the cart completely. It should:

- Validate that the user exists.
- Remove the specified flavor from the user's cart.
- Return the updated cart.

Example request:

```
{
  "userId": 3,
  "flavorId": 2
}
```

Example response:

```
{
  "success": true,
  "message": "Flavor removed from cart.",
  "cart": []
}
```

## 9. Place Order API

**Endpoint:** POST /orders

This endpoint converts the current cart into a saved order. It should:

- Validate that the user exists.
- Validate that the cart is not empty.
- Create a new order from the cart contents.
- Save the order to the user's orders list.
- Clear the user's cart after the order is placed.

Example request:

```
{
  "userId": 3
}
```

Example response:

```
{
  "success": true,
  "message": "Order placed successfully.",
  "orderId": 4
}
```

## 10. Order History API

**Endpoint:** GET /orders

This endpoint returns the logged-in user's previous orders. It should:

- Validate that the user exists.
- Return the user's saved order history.
- Return an empty list if the user has not placed any orders yet.

Example request:

```
GET /orders?userId=3
```

Example response:

```
{
  "success": true,
  "message": "Order history loaded.",
  "orders": [
    {
      "orderId": 4,
      "items": [
        {
          "flavorId": 2,
          "name": "Chocolate Bliss",
          "price": 5.49,
          "quantity": 2
        }
      ],
      "total": 10.98,
      "timestamp": "2026-03-30 18:30:00"
    }
  ]
}
```

## Frontend Pages and Navigation

Your updated application should include *Home*, *Flavors*, *Login*, *Signup*, and *Order History* pages.

The Home, Login, and Signup pages are public. The Flavors and Order History pages should require login. If a user attempts to access a protected page without being logged in, redirect them to the Login page.

## Login State

For this assignment, use simple frontend-held login state. After a successful POST /login, store the returned userId and username in localStorage. Use that stored login state to guard protected

pages. Add a logout button in the header, aligned to the right end. When clicked, it only needs to clear the frontend login state and redirect the user to the Home page.

# Page Requirements

## 1. Header and Footer

Reuse the existing Sweet Scoop header and footer style from Assignment 3.

*Requirements:*

- Update the navigation bar to include a link to the Order History page.
- Place the authentication control at the right end of the header, separate from the main navigation links.
- When no user is logged in, show a Login button in that location. The Login button should take the user to the Login page when clicked.
- When a user is logged in, replace the Login button with a Logout button in the same location.
- The Logout button should clear the login data in localStorage and redirect the user to the Home page.

## 2. Home Page

Retain the page structure from assignment 3. However, the displayed data must now come from the backend.

*Requirements:*

- Fetch flavor data from the backend and display 3 featured flavors (GET /flavors).
- Fetch reviews from the backend and display 2 random reviews (GET /reviews).

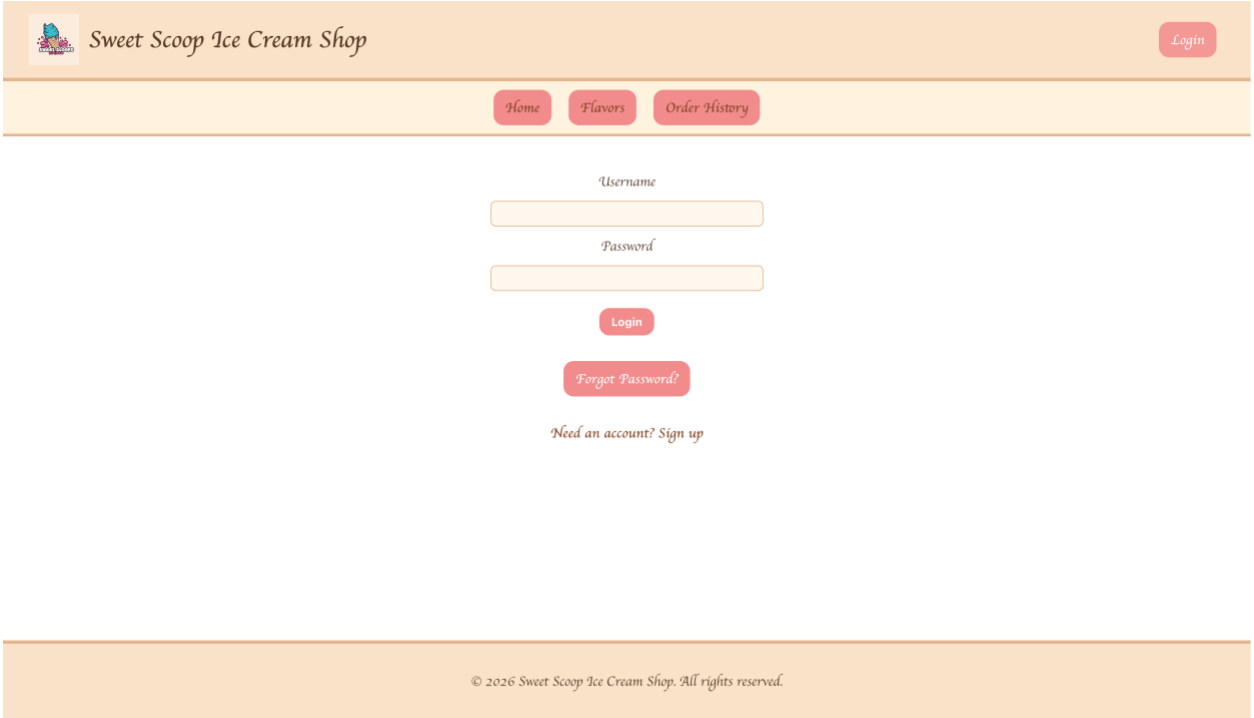
## 3. Login Page

Update the existing Login page from Assignment 3 so that it no longer uses the JSONPlaceholder API for validating users.

*Requirements:*

- Include a link or button that takes the user to the Signup page.
- Submit login credentials to the Flask backend using POST /login.
- Display a success or error message returned by the backend.
- On successful login, store the returned login state on the frontend and redirect the user to the Flavors page.

Example Login page:




*Login page example*

## 4. Signup Page

Add a new Signup page that matches the style of the Login Page. This page should include a form that collects a username, email, password, confirm password, and a signup button.

*Requirements:*

- Use the same validation rules described in the Registration API section for username, email, and password.

*You can test that your backend validation works by disabling the frontend validation and trying to submit invalid data. It is important to have validation on both the frontend and backend to ensure a good user experience while also maintaining security and data integrity.*

- Use POST /signup to submit valid signup data to the backend.
- Validate that confirm password matches password.
- Show clear error messages when validation fails or the username/email is already taken.
- On successful signup, redirect the user to the Login page.

Example Signup page:

*Signup page example*

## 5. Flavors Page

Update the existing Flavors page so that it works with the backend instead of local frontend state.

*Requirements:*

- Fetch the flavor catalog from GET /flavors.
- Fetch the logged-in user's cart from GET /cart.
- The data from the API should replace localStorage as the source of truth for the cart.
- Add a Place Order button to the page. When clicked, it should send a POST /orders request for the logged-in user.
- If the order is placed successfully, the backend should save it to the user's order history and clear the cart.
- Display any success or error message returned by the backend.

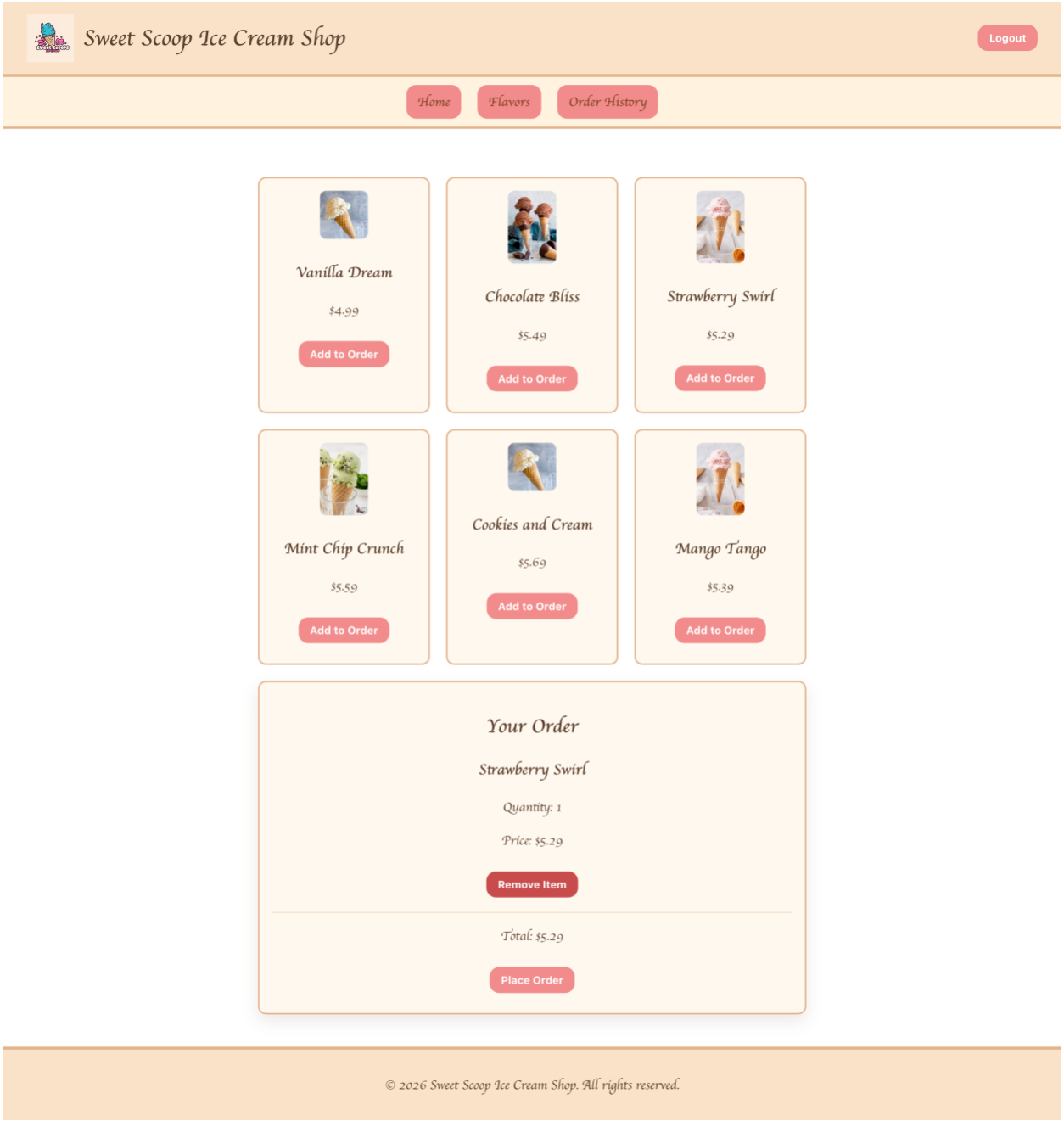
*Note: Images can still be served from the frontend public assets, so the image paths in the backend flavor data should point to those frontend asset locations.*

*Cart behavior*

- Clicking Add to Order should:
  - call POST /cart if the flavor is not already in the cart
  - call PUT /cart with the new quantity if the flavor is already in the cart

- Clicking Remove Item should remove the item from the cart completely using DELETE /cart.
- When the page loads or refreshes, repopulate the order list using GET /cart

Example Flavors page:




*Flavors page example*

## 6. Order History Page

Add a new Order History page. This page should display the logged-in user's previous orders in a clear and organized manner.

*Requirements:*

- Fetch the logged-in user's previous orders using GET /orders.
- Display each order clearly, including an order identifier, ordered items, total price, and order date or timestamp.
- If the user has not placed any orders yet, display a suitable message.

Example Order History page:

*Order history page example*